# Programming in Python
# IDC-212
# Lecture-2

August 5, 2025

# Contents

**1. Data Type**
1.1 List

# Data Type

# List Data Type (1/3)

**When to use List Data Type?**

- To represent a group of values as a single entity.
- When insertion order must be preserved.
- When duplicates are allowed.
- When heterogeneous objects are required.
- Lists are growable in nature.

**Properties of Lists:**

1. Insertion order is preserved.
2. Heterogeneous objects are allowed.
3. Duplicates are allowed.
4. Growable in nature.
5. Values should be enclosed in square brackets `[ ]`.

**Example:**

```python
list1 = [10, 10.5, 'iiser', True, 10]
print(list1)    # [10, 10.5, 'iiser', True, 10]
```

# List Data Type (2/3

**Indexing and Slicing:**

```python
list1 = [10, 20, 30, 40]
print(list1[0])      # 10
print(list1[-1])     # 40
print(list1[1:3])    # [20, 30]

list1[0] = 100
for i in list1:
    print(i)
# Output: 100, 20, 30, 40
```

# List Data Type (3/3)

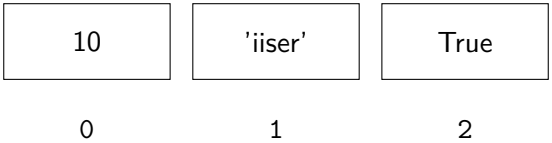
**Growable Nature:**

```
list2.remove(20) print(list2)  [10, 30, 'iiser']
list2 = list2 * 2 print(list2)  [10, 30, 'iiser', 10, 30, 'iiser']
```

**Note:** Lists are ordered, mutable, heterogeneous collections that allow duplicates.

# List Memory Layout Example

**Python List Example:** `list1 = [10, 'iiser', True]`




Each element can be accessed using its **index**.
Example: `list1[1]` $\Rightarrow$ `'iiser'`

# List Indexing with Positive and Negative Indices

**Example List:** `[10, "A", "B", 20, 30, 10]`

| -6 | -5 | -4 | -3 | -2 | -1 |
|---|---|---|---|---|---|
| 10 | A | B | 20 | 30 | 10 |
| 0 | 1 | 2 | 3 | 4 | 5 |

**Note:** List objects are **mutable**, i.e., we can change their content.

# Creation of List Objects (2/2)

### Creating a List with Dynamic Input

```python
list3 = eval(input("Enter List: "))
print(list3)
print(type(list3))
```

### Example Input:

```
Enter List: [1, 2, 'iiser', True]
```

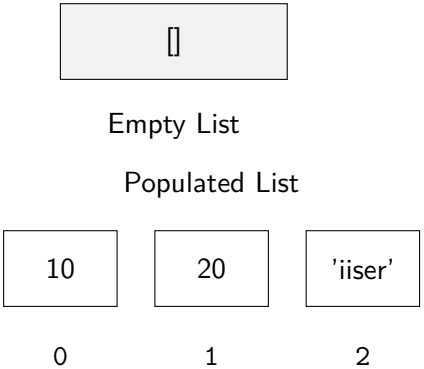
### Output:

```
[1, 2, 'iiser', True]
<class 'list'>
```

# Creation of List Objects with Memory Layout

**Example Lists**

- Empty list: `list1 = []`
- Populated list: `list2 = [10, 20, 'iiser']`



# Creating List Objects with list() Function

**Using list() with range():**

```
l = list(range(0, 10, 2))
print(l)
print(type(l))
```

**Output:**

```
[0, 2, 4, 6, 8]
<class 'list'>
```

# List Slicing Examples

**Syntax:** list2 = list1[start : stop : step]

- **start** → Index where slice begins (default = 0)
- **stop** → Index where slice ends (default = len(list))
- **step** → Increment value (default = 1)

```python
n = [1,2,3,4,5,6,7,8,9,10]

print(n[2:7:2])    # [3, 5, 7]
print(n[4::2])     # [5, 7, 9]
print(n[3:7])      # [4, 5, 6, 7]
print(n[8:2:-2])   # [9, 7, 5]
print(n[4:100])    # [5, 6, 7, 8, 9, 10]
```

# Forward Slicing in Python

**Example Code:**

```python
n = [1,2,3,4,5,6,7,8,9,10]
print(n[2:7:2])
# Output: [3, 5, 7]
```

| -10 | -9 | -8 | -7 | -6 | -5 | -4 | -3 | -2 | -1 |
|---|---|---|---|---|---|---|---|---|---|
| 1 | 2 | 3 | 4 | 5 | 6 | 7 | 8 | 9 | 10 |
| 0 | 1 | 2 | 3 | 4 | 5 | 6 | 7 | 8 | 9 |

Output: [3,5,7]

# Reverse Slicing in Python

**Example Code:**

```python
n = [1,2,3,4,5,6,7,8,9,10]
print(n[8:2:-2])
# Output: [9, 7, 5]
```

| -10 | -9 | -8 | -7 | -6 | -5 | -4 | -3 | -2 | -1 |
|---|---|---|---|---|---|---|---|---|---|
| 1 | 2 | 3 | 4 | 5 | 6 | 7 | 8 | 9 | 10 |
| 0 | 1 | 2 | 3 | 4 | 5 | 6 | 7 | 8 | 9 |

Output: [9,7,5]

# List vs Mutability

**Key Idea:** Once we create a List object, we can **modify its content**. Hence, **List objects are mutable**.

```
n = [10,20,30,40]
print(n)        # Output: [10, 20, 30, 40]

n[1] = 777
print(n)        # Output: [10, 777, 30, 40]
```

## Explanation

- Lists in Python are mutable. - After creating a list, we can change its elements by index. - Example: The second element (20) was replaced with 777.

# Traversing a List using `while` Loop

**Definition:** The sequential access of each element in a list is called **traversal**.

**Using `while` Loop:**

```python
numbers = [5, 10, 15, 20, 25]
index = 0
while index < len(numbers):
    print(numbers[index])
    index += 1
```

**Output:**

5
10
15
20
25

# Traversing a List using for Loop

**Using for Loop:**

```python
fruits = ["apple", "banana", "cherry", "mango"]
for item in fruits:
    print(item)
```

**Output:**

```
apple
banana
cherry
mango
```

**Comparison**

- while loop requires manual index handling. - for loop directly accesses each element.

# Important List Functions (1/2)

**1) len():** Returns the number of elements in the list.

```
numbers = [12, 24, 36, 48, 60]
print(len(numbers))   # Output: 5
```

**2) count():** Returns the number of occurrences of a specified item.

```
marks = [80, 90, 80, 75, 90, 80]
print(marks.count(80))   # Output: 3
print(marks.count(75))   # Output: 1
print(marks.count(100))  # Output: 0
```

# Important List Functions (2/2)

**3) index():** Returns the index of the first occurrence of the specified item.

```python
cities = ["Delhi", "Paris", "London", "Paris", "Tokyo"]

print(cities.index("Paris"))   # Output: 1
print(cities.index("Tokyo"))   # Output: 4

# Before using index(), check if element exists
print("New York" in cities)    # Output: False
```

**Note**

If the element is not in the list, `index()` raises a `ValueError`. Always check membership with `in` before using `index()`.

# Manipulating List Elements (1/2)

**1) append() Function:** Adds an element to the end of the list.

```
fruits = []
fruits.append("Mango")
fruits.append("Apple")
fruits.append("Orange")
print(fruits)
```

**Output:** ['Mango', 'Apple', 'Orange']

**Example:** Add all numbers up to 50 divisible by 7

```
multiples = []
for i in range(51):
    if i % 7 == 0:
        multiples.append(i)
print(multiples)
```

**Output:** [0, 7, 14, 21, 28, 35, 42, 49]

# Manipulating List Elements (2/2)

**2) insert() Function:** Inserts an element at a specified index.

```python
numbers = [11, 22, 33, 44]
numbers.insert(2, 99)   # insert at index 2
print(numbers)
```

**Output:**

```
[11, 22, 99, 33, 44]
```

**Another Example:**

```python
nums = [5, 15, 25]
nums.insert(0, 100)    # at the beginning
nums.insert(-1, 200)   # before the last element
print(nums)
```

**Output:**

```
[100, 5, 15, 200, 25]
```

# List Function: `remove()`

**Purpose:**

- Removes the **first occurrence** of a specified item from the list.
- If the element is not found, Python raises a `ValueError`.

**Example:**

```python
n = [5, 10, 15, 10, 20]
print(n)            # [5, 10, 15, 10, 20]
n.remove(10)
print(n)            # [5, 15, 10, 20]
if 100 in n:
    n.remove(100) # This won't execute
else:
    print("100 not found in list")
```

**Note:** Use the `in` operator before calling `remove()` to avoid errors.

# List Function: pop()

**Purpose:**

- Removes and returns an element from the list.
- By default, removes the last element.
- You can specify an index to remove an element from that position.

**Examples:**

```python
n = [10, 20, 30, 40]
print(n.pop())     # 40
print(n)           # [10, 20, 30]

print(n.pop(1))    # 20
print(n)           # [10, 30]

empty = []
# print(empty.pop()) -> IndexError: pop from empty list
```

**Note:** pop() is commonly used for stack operations (LIFO: Last In First Out).

# List Function: `reverse()`

**Purpose:**

- Reverses the order of elements in a list.
- It modifies the original list in place.

**Example:**

```python
numbers = [11, 25, 37, 49]
numbers.reverse()
print(numbers)   # [49, 37, 25, 11]
```

**Note:** Unlike slicing `[::-1]`, `reverse()` does not create a copy but changes the list itself.

# List Function: sort()

**Purpose:**

- Sorts the list in place according to natural ordering.
- For numbers: ascending order by default.
- For strings: alphabetical order by default.

**Examples:**

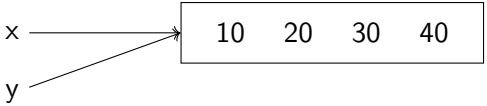
```python
nums = [42, 7, 19, 3, 15]
nums.sort()
print(nums)   # [3, 7, 15, 19, 42]

fruits = ["Mango", "Orange", "Apple", "Banana"]
fruits.sort()
print(fruits) # ['Apple', 'Banana', 'Mango', 'Orange']
```

**Note:** Use `sort(reverse=True)` for descending order.

# Aliasing of List Objects

Aliasing is the process of giving another reference variable to the same list.

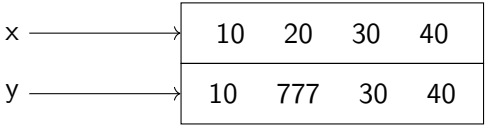
```python
x = [10, 20, 30, 40]
y = x
print(id(x))
print(id(y))
y[1] = 777
print(x)  # Output: [10, 777, 30, 40]
```



# Cloning of List Objects

Cloning creates a duplicate independent object using slicing or copy().

```
x = [10, 20, 30, 40]
y = x[:]    # or y = x.copy()
y[1] = 777
print(x)  # Output: [10, 20, 30, 40]
print(y)  # Output: [10, 777, 30, 40]
```



# Membership Operators in Python

- Membership operators check if an element exists in a list:
  1. **in** operator
  2. **not in** operator

```python
n = [5, 15, 25, 35]
print(15 in n)
print(50 in n)
print(25 not in n)
print(100 not in n)
```

# Nested Lists

We can take one list inside another list. Such lists are called **nested lists**.

**Example**

```python
n = [5, 15, [25, 35]]
print(n)          # [5, 15, [25, 35]]
print(n[0])       # 5
print(n[2])       # [25, 35]
print(n[2][0])    # 25
print(n[2][1])    # 35
```

**Note:** Nested lists can be accessed using index notation similar to multi-dimensional arrays.

# Nested List as Matrix

**In Python, we can represent matrices using nested lists.**

## Example

```python
matrix = [[2,4,6],[8,10,12],[14,16,18]]
print(matrix)

print("Row-wise output:")
for row in matrix:
    print(row)

print("Matrix style output:")
for i in range(len(matrix)):
    for j in range(len(matrix[i])):
        print(matrix[i][j], end=' ')
    print()
```

# Output

**Output:**

[2, 4, 6 , [8, 10, 12], [14, 16, 18]]

- Row-wise output:

2, 4, 6
8, 10, 12
14, 16, 18

- Matrix style output:
  - 2 4 6
  - 8 10 12
  - 14 16 18

# List Comprehensions

**List Comprehensions** provide an easy and compact way to create lists from any iterable (like List, Tuple, Dictionary, or Range) with optional conditions.

**Syntax:**

$$\text{list} = [\text{expression for item in iterable if condition}]$$

## Examples

```python
# Squares of numbers 1 to 8
squares = [x*x for x in range(1,9)]
print(squares)   # [1, 4, 9, 16, 25, 36, 49, 64]
# Powers of 3 for 1 to 5
powers = [3**x for x in range(1,6)]
print(powers)    # [3, 9, 27, 81, 243]1
# Even numbers from squares
evens = [x for x in squares if x % 2 == 0]
print(evens)     # [4, 16, 36, 64]
```